Weather Planner

CSU44000 Internet Applications – Assignment 1 Liliana Berenguer Jin

1. Requirements

Weather Planner is a small reactive Internet Application that takes a destination city and tells the traveller what to pack. The client runs in the browser using Vue 3. It talks to a Node.js / Express server, which does the actual work: it calls OpenWeather's APIs, reduces what comes back into a short summary, and returns that as one JSON object for the client to display.

Before writing any code, I listed the core requirements from the assignment and matched each one to the OpenWeather API that could provide the data. In retrospect this was a really useful starting point for deciding how to structure the server side. Table 1 shows more or less what I concluded from it.

Table 1 – Requirements and the OpenWeather API used for each

Requirement	OpenWeather API	What it provides
Turn the city name into a location	Geocoding (`/geo/1.0/direct`)	Latitude and longitude
Three-day table of temperature, wind speed and rainfall	5-day / 3-hour Forecast (`/data/2.5/forecast`)	Around 40 readings at three-hour intervals
Umbrella if rain is forecast; Cold / Mild / Hot packing advice (below 8 °C, 8–24 °C, above 24 °C)	Forecast (same response)	Rain and temperature per reading
Warning for each pollutant above the "Good" level, with its level and health risk	Air Pollution (`/data/2.5/air_pollution`)	AQI (1–5) and the concentration of each pollutant
Compare two cities side by side (extra feature)	All of the above, twice	Two summaries plus a verdict

The APIs themselves also set the order of the requests: the Forecast and Air Pollution APIs take coordinates rather than a city name, so Geocoding always has to go first.

2. System architecture

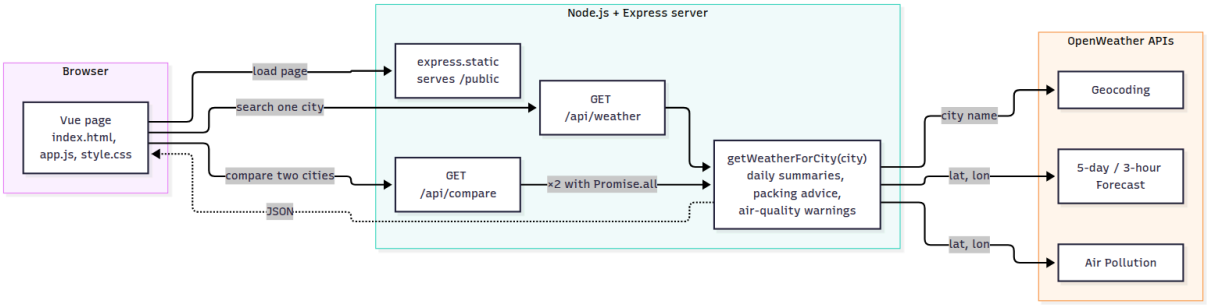


Figure 1 – System architecture. The browser loads the page from Express and then sends it a city through one of the two routes. Express calls `getWeatherForCity`, which asks OpenWeather for the location, the forecast and the air quality, builds the summary, and returns it to the browser as JSON. For a comparison the same function runs twice.

Some main observations:

- The route handlers only deal with HTTP. They read and validate the query string and turn errors into status codes. Everything else happens in `getWeatherForCity`, which fetches and processes the data for one city. Both routes call it, so when I added the comparison feature I did not have to duplicate the lookup. The helpers below it (`makeDailySummaries`, `makePackingAdvice`, `makeAirQualitySummary`) contain the rules and have no idea they are inside a web server.
- The server is stateless. Both routes are plain `GET` requests, there are no sessions and nothing is stored between calls, so the same request always gives the same answer. This was the simplest option and it also made the server easy to test on its own, by opening the route in the browser with a city in the URL.
- The client follows the MVC paradigm: the JSON from the server is the model, the HTML template is the view, and the Vue instance holding the data and methods is the controller. There is no manual DOM manipulation: the template describes what each state should look like and Vue updates the DOM when the data changes.
- I kept dependencies to a minimum. Node 22 already has `fetch` and can load `.env` itself, and Vue comes from a CDN, so the project only installs Express and nothing needs to be built before running it.

3. Implementation

3.1 Server side

I found myself building the server in three iterations, each one answering a different question: first, can I get anything out of OpenWeather at all; then, what do I do with what comes back; and finally, small refinements after finding bugs in the previous version.

Iteration 1: the first version just called the Geocoding and Forecast APIs and passed the response straight through, with nothing processed and no error handling. It did two things:

- Chain the two requests. The route reads the city from the query string, `await`s the Geocoding call to get the coordinates, then `await`s the Forecast call with them. While the handler waits on OpenWeather, the event loop is free to serve other requests, so this does not block JavaScript's only thread.
- Return the Forecast response as it comes, which was useful to see exactly what I was working with. OpenWeather does not return "days" but a `list` of about forty entries, one every three hours, each with its own temperature, wind and rain values, so showing three days meant grouping those entries myself.

Iteration 2: this is where the logic of the app got built on top of those responses:

- Daily summaries: the three-hourly entries are grouped by date, and for each day the server computes the min, max and average temperature, the average wind speed, the total rainfall and whether rain is expected.
- Packing advice: the Cold / Mild / Hot thresholds from the specification are applied to each day's average, and the advice becomes "pack layers" if the days fall into different categories. The umbrella flag, on the

other hand, is set if any day expects rain.

- Air quality: the Air Pollution API call was added. Each pollutant is compared with the upper limit of OpenWeather's "Good" band, and anything above it is reported with its band and a short health risk.
- Error handling: the route checks the city before doing anything, and every failure ends in an HTTP status code and a message the client can show (Table 2).

Status	Condition
200 (OK)	Success: `city`, `today`, `days[3]`, `packing`, `airQuality`
400 (Bad Request)	City missing or over 100 characters
404 (Not Found)	Geocoding returns no match
500 (Internal Server Error)	API key missing from `.env`
502 (Bad Gateway)	OpenWeather unreachable, returned an error, or returned too little data

Table 2 – Status codes returned by `GET /api/weather`. The client only checks `response.ok` and shows the message; the specific codes are there so the API also makes sense to anything other than this client.

Iteration 3: once the logic was in place I tried the app with more cities and at different times of day, and three things needed fixing:

- Days were grouped on UTC dates, which gave the wrong first day for cities far from Greenwich. Entries are now shifted by `city.timezone` first, and the current partial day is shown separately as "today".
- A failure in the Air Pollution request used to take the whole response down. It now has its own `try/catch`, so the forecast still renders if air quality is unavailable.
- The route handler had grown into one long `app.get`. The lookup moved into its own function, `getWeatherForCity`, which is what later made the comparison feature cheap to add.

3.2 Sequence diagram of a search

Figure 2 shows what the final server does for one search. The browser sends the city to `/api/weather`. The server calls Geocoding to turn the name into coordinates, then Forecast with those coordinates for the three-hourly readings, then Air Pollution with the same coordinates for the current air quality. The three calls run one after the other because each needs the result of the previous one, and the air-quality call is the only one allowed to fail. Once the data is in, the server builds the daily summaries, packing advice and pollutant warnings, and returns everything as one JSON object; the browser never sees the intermediate responses.

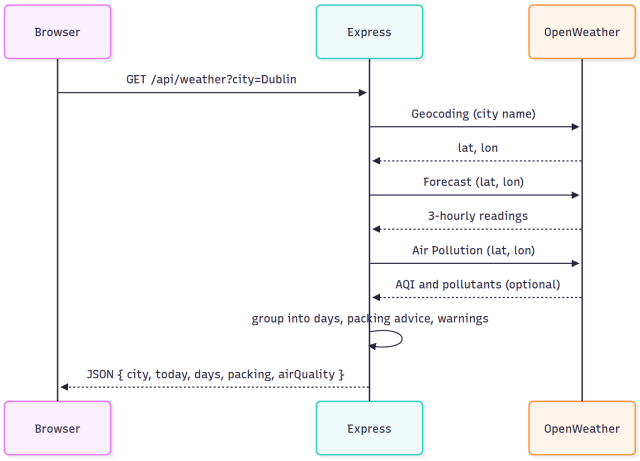


Figure 2 – One search handled by the final server

3.3 Client side

The client, on the other hand, was much more straightforward. Everything lives under `public/`: an `index.html` with the page structure and the Vue directives, an `app.js` with the Vue instance, and a separate `style.css`. I began with a skeleton that only had an input, a button and the raw JSON printed on screen, and then built the interface on top of that. The page has a small amount of state (the city typed in, whether a request is running, the result, and any error) and the HTML describes what to show for each of those. As in any reactive app, when the server replies the state changes and Vue updates the DOM for us (nothing needs to be updated by hand).

Within that, the HTML uses the usual Vue directives: `v-model` for the input, `v-if` to switch between the loading, error and result views, `v-for` for the forecast rows and pollutant warnings, and `:class` for the Cold / Mild / Hot colours. The JavaScript side is just the state, a method per form that sends the `fetch` request and stores the reply, and a few formatters.

For the layout, today's summary sits on the left and the forecast table, packing advice and air quality on the right, with an icon for each day's conditions.

Figure 3 – Result view for Dublin *(screenshot)*

3.4 Additional feature: comparing two destinations

Once a city is loaded, the user can enter a second one and see both forecasts side by side with a short verdict: which has the better weather, which needs an umbrella, and which has worse air. I chose this because deciding between two places is the most realistic reason to open a packing app.

- Server side: a new route, `/api/compare`, that receives two city names and calls the existing `getWeatherForCity` for each. Both calls return promises, so instead of waiting for one city before starting the other, the route starts both and waits for the pair with `Promise.all`. The two results are then compared: the city with fewer rainy days wins, more Mild days breaks a tie, and the worse air-quality index is flagged.
- Client side: a second input that appears under the first result. The two summaries are rendered next to each other with the winner marked, and the comparison has its own loading and error state so that a failed comparison does not affect the forecast already on screen.

Figure 4 – Dublin compared with Lisbon *(screenshot)*

4. Testing and next steps

Testing was done by hand, using cities and situations that cover the different paths through the code:

- Dublin, IE as the normal case, and Paris against Paris, US to make sure the country code is being used.
- A made-up city name and a missing API key, which should produce the 404 and 500 responses with their messages.
- Tokyo, where the day grouping only comes out right if the local timezone is applied.
- The Air Pollution request with a wrong URL, to see the forecast still render without it.
- A comparison where one of the two cities is invalid.

For a future version there are a few things I would add:

- Make use of the other Geocoding matches, so the user can pick the right city instead of having to type a country code.
- Use OpenWeather's air-pollution forecast rather than the current reading, so the warning covers the days of the trip.
- Deploy the server on AWS so the app does not have to run locally, and cache OpenWeather responses for a few minutes, since a comparison currently costs six calls.

Appendix A – Source code · **Appendix B – Academic integrity declaration**